서울시립대학교
UNIVERSITY OF SEOUL

전자전기컴퓨터설계실험Ⅱ

실험 후 보고서

LAB6

학과	전자전기컴퓨터공학부
이름 / 학번	김승민 / 2023440021
작성일	2026년 11월 1일
대상	UART 통신과 FPGA LED 제어
검증 환경	VS Code / Vivado / STM32 / FPGA

1. 실험 목적과 관련 이론

실험 목적과 결과 개요

이번 실험에서는 PC에서 입력한 문자를 STM32가 수신하고, 이를 FPGA에 전달하여 수신 내용과 LED 제어 결과를 확인하였다. 과제 5의 문자·16진수 출력, 과제 6의 장치 간 문자 전달, 과제 7의 LED 켜짐·꺼짐을 실험 사진으로 확인하였다.

FPGA 개별 실습에서는 SRAM 읽기·쓰기에 성공하였다. 서보모터와 DAC는 이번 주에 정상 동작을 구현하지 못했으며, 다음 주 재실험 결과를 실험 후 보고서에 추가할 예정이다.

UART 통신과 바이트 표현

UART는 공통 클록선 없이 송수신 속도를 맞추어 데이터를 주고받는 비동기 직렬 통신이다. 본 실험에서는 9600 bps, 데이터 8비트, 패리티 없음, 정지 비트 1개인 8N1을 사용하였다. 한 프레임은 시작 비트 1개, 데이터 8개, 정지 비트 1개로 구성된다. 문자와 수치 자체를 구분해야 한다. 문자 1의 ASCII 값은 0x31이고, 수치 1을 한 바이트로 표현한 0x01과 다르다. 대문자 A와 소문자 a도 각각 0x41과 0x61로 구별된다. STM32는 받은 바이트를 변환하지 않고 UART3로 전달하며, FPGA는 수신 바이트를 ASCII 명령과 비교한다.

STM32 송신 완료와 종단 수신의 구분

HAL_UART_Transmit은 지정한 UART로 데이터를 보내는 함수이다. 반환값 HAL_OK는 송신 함수가 정상적으로 끝났다는 의미이며, 상대 FPGA가 같은 문자를 수신했다는 응답은 아니다. 따라서 송신 로그와 별도로 FPGA의 USB-UART 출력 및 실제 LED 상태를 확인하였다.

2. 회로 구성과 실험 방법

장치와 통신 설정

항목	설정 및 역할
MCU	NUCLEO-G474RE
FPGA	HBE-Combo II-DLD
PC 연결	STM32 ST-LINK VCP / FPGA USB to Serial
공통 통신	9600 bps / 8N1 / 흐름 제어 없음
전송 방법	줄바꿈 None, 문자 단위 전송
개발 환경	VS Code / Vivado

과제 5는 PC와 STM32만 연결하여 수행하였다. 과제 6과 7에서는 송신 PC에 STM32를 연결하고, FPGA의 USB to Serial 단자를 다른 PC에 연결하여 수신 화면을 관찰하였다. 두 PC가 서로 달라도 각각 올바른 COM 포트와 통신 속도를 선택하면 된다. 실험 사진에서 송신 측은 COM4, 수신 측은 COM6으로 나타났다. COM 번호는 해당 PC의 장치 할당값이다.

과제 6과 7의 배선

연결 대상	실제 신호 경로
MCU TX	PC10 USART3_TX → PORT A I/O 00
FPGA RX	BDIO_00 / FPGA 패키지 F8
공통 기준	STM32 GND ↔ FPGA GND
FPGA TX	F6 USB_TXD → 보드 USB-UART → 수신 PC
제어 LED	led[7] / L4 / 보드 LED1

최종 XDC는 uart_rxd를 F8에 배정하였다. 이는 교재의 PORT A I/O 00 핀 구성표와 대응하며, 커넥터 이름 EX3 또는 신호명 EXT3와 구분하였다. PORT A ENABLE은 사용한 제약 파일의 안내에 따라 OFF로

두어 버스 스위치의 활성 LOW 연결을 사용하였다. FPGA는 3.3 V 논리 입력을 사용하며, 이번 연결에는 9핀 RS232 단자를 사용하지 않았다.

```
set_property PACKAGE_PIN F8 \
  [get_ports uart_rxd]
set_property PULLUP true \
  [get_ports uart_rxd]
```

3. 과제 5 PC와 MCU 입출력

설계 방법과 핵심 코드

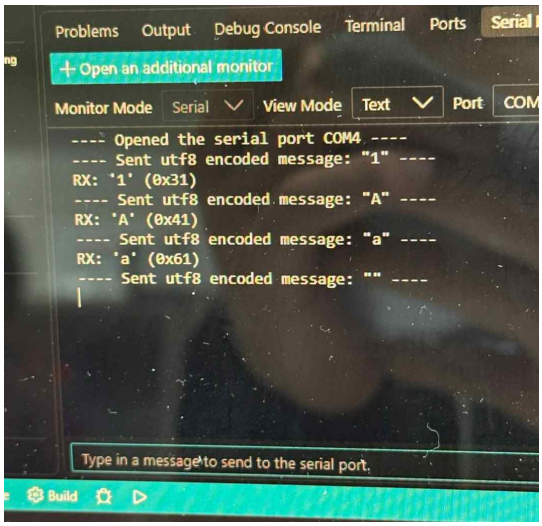
표준 입력 getchar로 받은 문자를 int에 저장하여 EOF와 구분하였다. 정상 입력은 uint8_t로 변환하고, 출력 가능한 문자는 문자와 16진수 값으로 표시하였다. 표준 입출력은 USART2에 연결되며, 입력이 없으면 getchar에서 기다리는 방식으로 동작한다. 이번 코드는 같은 USART2에 인터럽트 수신을 동시에 시작하지 않는다.

```
int ch = getchar();
if (ch == EOF) {
    clearerr(stdin);
    return;
}
uint8_t data = (uint8_t)ch;
printf("RX: '%c' (0x%02X)",
       (int)data,
       (unsigned int)data);
```

위 코드는 실제 소스의 입력 처리와 문자 출력 부분을 발췌한 것이다. 제어 문자에 대한 분기 및 상태 카운터 등 전체 구현은 첨부 소스에 포함하였다.

실험 결과와 예상값 비교

입력	예상값	실제 출력
1	0x31	'1' (0x31)
A	0x41	'A' (0x41)
a	0x61	'a' (0x61)



위의 그림에서 세 문자의 출력이 예상한 ASCII 값과 모두 일치하였다. 특히 A와 a의 출력값이 다르므로 대소문자를 유지하여 수신했음을 확인하였다. 문자 1 역시 0x01이 아니라 0x31로 출력되어 문자 인코딩을 올바르게 해석하였다.

실험 결과 동영상은 github에 업로드하였다.

https://github.com/thechef030/STM32_2/tree/stm32-2/evidence/board

4. 과제 6 MCU에서 FPGA로 전달

설계 방법과 핵심 코드

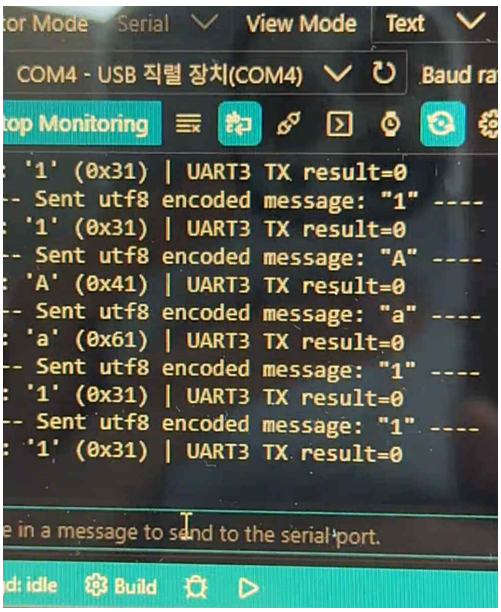
과제 5에서 수신한 바이트를 UART3로 한 번 전송하도록 확장하였다. STM32는 PC10으로 데이터를 송신하고, FPGA는 PORT A 입력에서 UART 프레임을 복원한 뒤 USB-UART 송신 회로를 통해 수신 PC로 전달한다. MCU가 FPGA 응답을 되받는 방식은 아니며, 최종 수신 PC의 화면에서 전달 결과를 확인한다.

```
HAL_StatusTypeDef result =
    HAL_UART_Transmit(
        &huart3, &data, 1U, 100U);
printf(" | UART3 TX result=%d",
       (int)result);
```

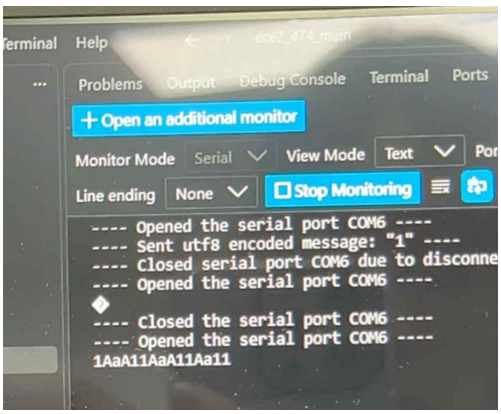
FPGA 수신 회로는 비동기 입력을 동기화한 뒤 시작 비트를 확인하고, 각

데이터 비트의 중앙 부근에서 LSB부터 샘플링한다. 정지 비트가 HIGH인 정상 프레임에 대해 rx_valid를 발생시킨다. 수신 바이트는 16바이트 FIFO를 거쳐 송신되며, UART TX는 시작 비트·데이터·정지 비트 순으로 출력한다.

송신 및 수신 결과



위의 그림에서 1, A, a를 입력했을 때 각각 올바른 문자와 바이트 값이 출력되고 UART3 TX result=0이 표시되었다. 이는 해당 송신 호출이 HAL_OK로 완료되었다는 것을 확인하는 결과이다.



위의 그림의 하단에는 1, A, a가 포함된 연속 문자열이 표시되어 FPGA를 거친 문자 전달을 확인하였다. 줄바꿈을 넣지 않았으므로 수신 문자는 한 줄에 이어서 표시된다.

실험 결과 동영상은 github에 업로드하였다.

https://github.com/thechef030/STM32_2/tree/stm32-2/evidence/board

5. 과제 7 UART 명령으로 LED 제어

설계 방법과 명령 해석

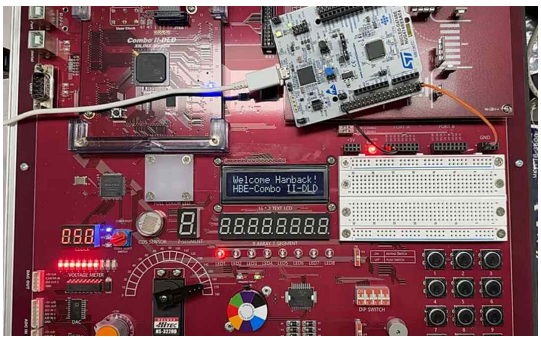
MCU의 전달 경로는 과제 6과 동일하게 사용하고, FPGA 수신 처리에 LED 명령 해석을 추가하였다. rx_valid가 발생할 때만 LED 상태를 갱신한다. 문자 0은 끄기, 1은 켜기, 2는 반전이며, 다른 바이트는 기존 상태를 유지하도록 구현하였다.

```
if (rx_valid) begin
    case (rx_data)
        8'h30: led_on <= 0;
        8'h31: led_on <= 1;
        8'h32: led_on <= ~led_on;
        default:
            led_on <= led_on;
    endcase
end
```

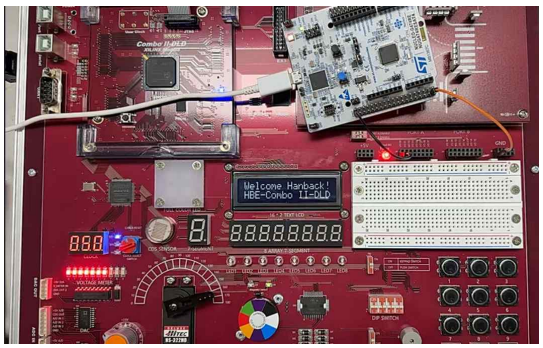
실제 출력은 led[7]을 물리 LED1에 연결하고 나머지 LED 출력은 LOW로 두었다. ASCII 문자를 직접 비교하므로 문자 1을 수치 1로 변환하지 않는 MCU 처리와 일치한다.

보드 관찰 결과

- LED 점등



- LED 비점등



위의 그림에서는 LED 배열의 첫 번째 LED1이 켜져 있고, 다음 그림에서는 같은 LED가 꺼져 있다. 보드 전원 표시등과 LED1을 구분하여 관찰했으며, 과제 7의 켜짐·꺼짐 상태가 확인되었다.

실험 결과 동영상은 github에 업로드하였다.

https://github.com/thechef030/STM32_2/tree/stm32-2/evidence/board

6. SRAM 읽기와 쓰기

SRAM 설계 방법

SRAM 실습은 주소에 데이터를 기록하고 같은 주소에서 읽어 저장값이 유지되는지 확인하는 것을 목표로 하였다. 실험 전 설계에서는 18비트 주소 버스 중 하위 4비트를 사용하여 주소 0~15를 선택하고, 16비트 데이터 버스의 하위 4비트를 입력 및 LED 표시에 사용하였다. 쓰기 중에는 FPGA가 버스를 구동하고, 읽기 중에는 FPGA 출력을 Z 상태로 두어 SRAM이 버스를 구동하도록 하였다.

```
assign drive_data =
    ctrl_w && !ctrl_r;
assign read_data =
    ctrl_r && !ctrl_w;
assign ram_cs =
    !(drive_data || read_data);
assign ram_oe = !read_data;
assign ram_we = !drive_data;
```

위 코드는 사전 보고서에서 사용한 SRAM 버스 제어의 핵심이다. 읽기와 쓰기 요청이 동시에 들어오면 두 동작을 모두 비활성화하여 데이터 버스의 동시 구동을 피하도록 구성하였다. 입력 주소를 확장한 ram_address와 읽기 데이터의 LED 연결은 전체 소스에서 확인할 수 있다.

SRAM 실험 결과

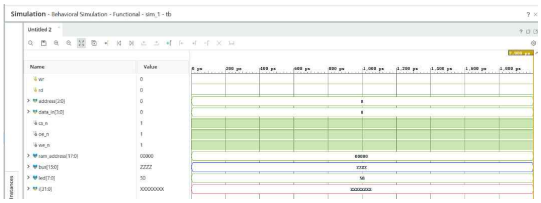
실험 결과가 사진으로 표현되지 않아 실험 결과를 github에 업로드하였다.

https://github.com/thechef030/STM32_2/tree/stm32-2/evidence/Vivado

SRAM bitstream 생성



SRAM Vivado 파형



위의 vivado simulation을 보면 0~2000 ps, 즉 0~2 ns 구간이다. wr=0, rd=0이며 cs_n·oe_n·we_n은 1, bus는 ZZZZ로 표시되어 비활성 상태의 버스 해제를 볼 수 있다. 주소와 입력 데이터는 0이고 led는 0x50, 반복 변수 i는 X로 나타난다.

읽기·쓰기 전환이 보이지 않으므로 이 화면만으로 모든 주소의 저장·읽기 검증이 끝났다고 판단하지 않는다. 실험 성공은 실제 보드 수행 결과이며, 이 초기 파형과 구분하여 기록하였다.

7. 서보모터 작동

서보모터 실험 결과

서보모터는 펄스 폭에 따라 위치가 변하는 것을 확인하는 실습이지만, 이번 실제 보드

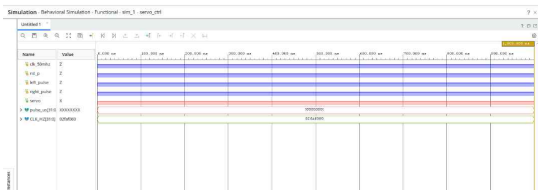
실험은 실패하였다. 비트스트림 생성까지 진행한 결과와 확인이 필요한 항목을 다음과 같이 정리하였다.

서보모터 bitstream 생성



비트스트림 생성은 완료되었으나 WNS가 −0.520 ns로 표시되었다. 따라서 설정된 제약에 대한 setup 타이밍을 충족하지 못한 상태이다. 구현 자원은 LUT 451개와 FF 73개이며, Failed Routes는 0, Methodology 경고는 4개이다. 타이밍 미충족은 다음 실험 전에 확인할 항목이지만, 이 화면만으로 보드 실험 실패의 직접 원인이라고 단정할 수는 없다.

서보모터 vivado 파형



위의 Vivado 파형은 0~1000 ns 구간이다. clk_50mhz, rst_p, left_pulse, right_pulse가 모두 Z(고임피던스)로 표시되어 입력이 구동되지 않은 상태이며, servo와 pulse_us는 X(미정값)이다. 따라서 정상적인 PWM 출력이나 펄스 폭을 확인할 수 없다. CLK_HZ=0x02FAF080은 50 MHz 설정값으로, 실제 클록이 인가되었다는 의미는 아니다. 다음 실험에서는 테스트벤치에서 클록·리셋·버튼 입력을 구동하고, 20 ms 프레임을 관찰할 수 있도록 시뮬레이션 시간을 늘려 검증할 예정이다.

8. DAC

DAC 실험 결과

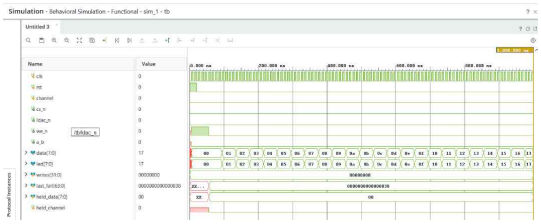
DAC 실습은 디지털 입력값에 따른 아날로그 출력의 변화를 확인하는 것이 목표였으나, 이번 실제 보드 실험은 실패하였다. 시뮬레이션의 디지털 데이터 변화와 보드에서의 아날로그 출력 성공은 구분하였다.

DAC bitstream 생성



비트스트림 생성 완료와 WNS 14.361 ns, TNS 0.000 ns, WHS 0.224 ns, THS 0.000 ns를 확인하였다. 구현 자원은 LUT 27개와 FF 63개, Failed Routes는 0, Methodology 경고는 20개이다. 표시된 타이밍 항목이 비음수이더라도 외부 DAC의 제어 조건과 실제 아날로그 출력까지 검증되었다는 의미는 아니다.

DAC vivado 파형



DAC vivado simulation을 보면 0~1000 ns 구간에서 data와 led가 0x00부터 0x17까지 증가하는 것이 보인다. 그러나 we_n은 초기 전환 이후 LOW로 유지된 것으로 표시되고 writes는 0이다. 따라서 반복 쓰기 완료를 이 화면으로 확인할 수 없으며, 데이터 증가만으로 DAC 동작 성공을 판단하지 않는다. 다음 주에는 쓰기 신호의 연결과 펄스 폭, 데이터 유지 시간, 실제 출력 측정 지점을 점검하고 DAC 실험을 이어서 수행할 예정이다.

9. 문제 해결 과정과 고찰, 결론

FPGA 구현 결과 요약

비트스트림 생성 완료는 실제 주변장치 실험 성공과 별개의 단계이다. 서보모터와 DAC는 다음 주에 계속 실험하고, 보완한 코드와 계측 결과를 다음 주 보고서에 추가한다.

물리 연결과 제약 파일의 일치

초기에는 보드 내부 USB-UART 입력과 확장 GPIO 입력, 커넥터 명칭을 구분하는 과정이 필요하였다. 교재의 핀 구성표를 확인한 뒤 최종적으로 PORT A I/O 00을 선택하고, uart_rxd의 패키지 핀을 F8로 지정하였다. 내부 USB-UART 입력 C6를 그대로 사용하면 STM32에서 외부 점퍼선으로 보낸 신호를 해당 수신 회로에 넣을 수 없으므로, 실제 배선과 XDC의 일치가 중요하였다.

실험 결론

PC–STM32 통신에서 문자와 ASCII 바이트 값의 대응을 확인하고, UART3와 FPGA 수신·송신 회로를 거쳐 다른 PC로 데이터를 전달하였다. FPGA LED1의 점등과 소등을 관찰하여 UART 입력을 외부 장치 제어로 확장할 수 있음을 확인하였다. 송신 함수의 정상 반환만으로 전체 통신 성공을 판단하지 않고 최종 수신 결과를 함께 확인해야 한다는 점을 배웠다.

SRAM은 기본 읽기·쓰기에 성공했지만, 서보모터와 DAC는 완료하지 못하였다. 다음 주에는 두 실습의 제어 파형과 실제 출력을 다시 확인하여 실패 원인 및 보완 결과를 기록할 예정이다.

11. GitHub 저장소와 버전

github 주소:
https://github.com/thechef030/STM32_2
브랜치: stm32-2
커밋:
2a67ff15f065a495d1b83d0116a1510965a28772

12. 참고문헌

전전설2 LAB6 교안